

Code Checklist	
Project Name	ToDdList-APP
Reviewed Document	Code review
Reviewed Feature/Scope	Backend + Frontend only (exclude mobile), checklist items 1-60
Number of Pass	78
Number of Fail	85
Number of N/A	5
Number of Errors	76

No.	Checklist Question	Review Result (Pass/Fail/N/A)			Note
		Pass	Fail	N/A	
1	Has the coding guideline specified in the project plan been adhered to?	☒	☒	☐	No project plan/coding guideline document was found in the reviewed backend/frontend scope, so adherence cannot be verified; 'backend/package.json' also has no lint/format script to enforce a coding standard.
2	Is inline documentation adequate?	☐	☒	☐	'backend/src/services/task.service.js' (~3358 lines) and 'frontend/app/components/views/TasksView/TaskDetailModal.tsx' (~2665 lines) contain complex logic with limited inline explanation, so inline documentation is not adequate for the current complexity.
3	Do naming conventions conform to the configuration management plan?	☐	☒	☐	No configuration management plan or naming convention document was found in the reviewed scope; naming is also inconsistent across source, for example model files 'Task.model.js'/'User.model.js' use PascalCase while controller files such as 'admin.controller.js'/'auth.controller.js' use lowercase naming without a documented convention.
4	Has code been properly formatted?	☐	☒	☐	Formatting/readability is inconsistent. 'frontend/app/components/common/UserMenu.tsx' and 'frontend/app/constants/groupRoles.ts' contain garbled UTF-8 text in comments/labels, and the backend has no formatter script/config in 'backend/package.json' to enforce consistent formatting.
5	Has a common set of routines been written without duplicating these routines in different programs?	☐	☒	☐	Common routine duplication exists in frontend: 'getInitials()' is duplicated in 'frontend/app/components/common/UserMenu.tsx' and 'frontend/app/components/ProfileSettings.tsx' instead of being shared as a reusable helper.
6	Is there any redundant or trash code?	☐	☒	☐	Redundant/debug code remains in the reviewed scope: 'frontend/app/debug-api.js' is a debug-only source file, and 'frontend/app/services/task.service.ts' contains extensive request/response debug 'console.log' statements that are not part of core business logic.
7	Are there any declared routes, handlers, or helper functions that are never used?	☐	☒	☐	Unused helper found: 'getRoleSummary' is exported in 'frontend/app/utlis/groupRoleUtils.ts' but no usage was found in the reviewed backend/frontend scope.
8	Have pointers been set to NULL if necessary?	☐	☐	☒	Not applicable for JavaScript/TypeScript in this backend/frontend scope because the reviewed code does not use manual pointers or NULL pointer management.
9	Does pointer arithmetic result in pointing to memory that is out of range?	☐	☐	☒	Not applicable for JavaScript/TypeScript in this backend/frontend scope because the reviewed code does not perform pointer arithmetic or direct memory access.
10	Are all the array indices within bounds?	☐	☒	☐	Array index bounds are not fully guarded in 'frontend/app/services/meeting.service.ts', where 'event.streams[0]' is accessed in two 'ontrack' handlers without checking that 'event.streams' contains at least one stream.
11	Are all the array indices correctly initialized?	☒	☐	☐	
12	Are all the branch conditions correct?	☐	☒	☐	Incorrect branch condition found in 'frontend/app/components/ProfileSettings.tsx': the field label is generated from 't('regional.timezone')', but the save branch checks 'field.label === 'Time Zone'', so the timezone-specific branch fails when the UI language is not English.
13	Do all loops terminate?	☒	☐	☐	
14	Is the condition for terminating a loop realistic?	☒	☐	☐	
15	Have the denominators in division operations been checked for zero before performing the division?	☒	☐	☐	
16	Can any statements placed inside a loop be placed outside the loop?	☐	☒	☐	In 'frontend/app/admin/page.tsx', 'const maxCount = Math.max(...analytics.activeUsersChart.map((d: any) => d.count), 1);' is recomputed inside 'analytics.activeUsersChart.map(...)' for every item even though it is invariant for the whole loop and can be moved outside.
17	Are there any portions in the code that the thread of execution never reaches?	☐	☒	☐	'frontend/app/utlis/groupRoleUtils.ts' defines 'canManageRoles = () => false', so the 'canEditRoles'-guarded edit/delete branches in 'frontend/app/components/views/GroupMembersView.tsx' are effectively unreachable during execution.
18	Are "if" statements nested to more than three levels?	☒	☐	☐	
19	Do the actual and formal interface parameters match?	☒	☐	☐	
20	Are there any unused variables declared?	☐	☒	☐	Running 'npm run lint' in the frontend reported many unused-variable warnings, for example 'setEmail' in 'frontend/app/components/ProfileSettings.tsx', 'canDeleteFolders'/'canEditFolders'/'canAssignFolders' in 'frontend/app/components/layouts/Sidebar.tsx', and 'Filter'/'selectedUser' in 'frontend/app/admin/page.tsx'.
21	Has the memory been correctly initialized?	☐	☐	☒	Not applicable for JavaScript/TypeScript in this backend/frontend scope because the reviewed code does not manually allocate or initialize raw memory buffers as part of normal application logic.

22	Has dynamic memory that has been allocated on entry been released at all exit points?	☐	☐	☒	Not applicable for the reviewed JavaScript/TypeScript code because memory lifecycle is handled by the runtime garbage collector rather than manual deallocation at each exit point.
23	Do MongoDB queries use appropriate indexes for frequently searched, filtered, or sorted fields?	☒	☐	☐	
24	Are database operations wrapped with proper error handling (try/catch), and are query failures logged or returned consistently?	☒	☐	☐	
25	Are concurrent update risks handled appropriately (atomic update, optimistic check, transaction where needed)?	☐	☒	☐	Multi-document write flows are not protected by a transaction or optimistic check in several places; for example 'backend/src/services/task.service.js' links two tasks by saving 'linkedTask' and 'sourceTask' separately, and 'backend/src/services/notification.consumer.js' accepts a group invitation by updating 'group', 'notification', and 'user' in separate saves, which can leave partial state if one step fails mid-flow.
26	Have the following conditions been checked in expressions?	☐	☒	☐	Partially only. The reviewed scope contains both good defensive checks and concrete misses, so the overall checklist item is marked Fail; see sub-items 26a-26e below.
	a. Are null or undefined values handled before the expression is evaluated?	☐	☒	☐	
	b. Are type mismatches prevented before comparison or calculation?	☒	☐	☐	
	c. Are default or fallback values applied where missing input may occur?	☒	☐	☐	
	d. Can the expression produce NaN, Infinity, or another invalid result?	☐	☒	☐	there is at least one unguarded case. In 'frontend/app/components/common/VoiceMessagePlayer.tsx', 'const percentage = clickX / rect.width;' does not guard against 'rect.width === 0', so the expression can produce 'Infinity' or 'NaN' before it is applied to 'audio.currentTime'.
	e. Are user inputs or request values sanitized before being used in expressions?	☒	☐	☐	
27	Will the requirements of response time be met?	☐	☒	☐	No based on the project's own performance evidence. 'test-docs/performance/BAO_CAO_THUC_THI_PERFORMANCE_JMETER_2026-04-22.md' states the smoke run already had 'PERF-01 Login' at '2764ms' over a '2000ms' threshold, and the full run failed almost completely ('99.82%' error rate) with 'Read timed out' and 'Connection reset'.
28	Is there a better alternative for improving the response times?	☐	☒	☐	The performance report itself recommends better alternatives that are not yet reflected as completed improvements, including staged load reruns, MongoDB index review for login/task/notification/chat paths, and trimming/projection of large payloads for kanban, chat history, and admin lists.
29	Have the following checks been performed	☒	☐	☐	
	a. Checks for empty query results, missing documents, or empty files/uploads?	☒	☐	☐	
	b. Checks for I/O, database, file upload, or external service errors?	☒	☐	☐	
30	Are the error messages clear? Are the error messages adequate?	☐	☒	☐	Not consistently. Several source files still contain garbled UTF-8/Vietnamese text in user-facing strings, for example upload error responses in 'backend/src/middlewares/upload.js' and multiple labels/messages in frontend files such as 'frontend/app/components/common/UserMenu.tsx', which reduces clarity and adequacy of displayed errors.
31		☐	☒	☐	Some runtime edge cases are still not trapped before execution, for example 'frontend/app/services/meeting.service.ts' accesses 'event.streams[0]' without guarding the stream array length, and 'frontend/app/components/common/VoiceMessagePlayer.tsx' calculates 'clickX / rect.width' without guarding zero width.
	Have all error conditions been trapped and handled?				
32	In arithmetic expressions, have the following been addressed:	☐	☒	☐	Partially only. The reviewed scope contains both well-formed arithmetic and a few readability/safety issues, so the overall item is marked Fail; see sub-items 32a-32f.
	a. Is the order of processing unambiguous?	☒	☐	☐	
	b. Is there any need for horizontal scrolling to read the entire expression?	☐	☒	☐	Some inline arithmetic expressions are long enough to hurt readability, for example the width calculation in 'frontend/app/admin/page.tsx' that nests a ternary, 'Number(...)', division, multiplication, and 'Math.min(...)' inside one JSX style expression.
	c. Are all parentheses properly closed? Do they ensure proper order of processing?	☒	☐	☐	
	d. Is rounding off performed along with the expression?	☒	☐	☐	
	e. Is division joined with another expression?	☐	☒	☐	There are combined arithmetic expressions that chain division with additional operations, for example '(currentTime / duration) * 100' in 'frontend/app/components/common/VoiceMessagePlayer.tsx' and '(item.count / maxCount) * 100' in 'frontend/app/admin/page.tsx'.
	f. Does any expression use table fields or file fields directly in the expression?	☐	☐	☒	Not applicable as written for this JavaScript/TypeScript web application review; the checklist wording assumes table/file-field style procedural systems rather than the current object/document model.
	In relational expressions, have the following been addressed	☐	☒	☐	Partially only. Several relational checks are sound, but there are also concrete purpose/readability issues; see sub-items 33a-33d.
	a. Are comparisons between the same types of data?	☒	☐	☐	
	b. Is it possible to have more than two outcomes for any expression ?	☒	☐	☐	
	c.Does the expression serve the purpose for which it is used?	☐	☒	☐	No, not always. In 'frontend/app/components/ProfileSettings.tsx', the comparison 'field.label === 'Time Zone'' does not reliably serve its intended purpose once labels are translated, so the timezone branch becomes logically incorrect for non-English UI.

33	d. Is there any need for horizontal scrolling to read the entire expression?	☐	☒	☐	Some relational checks are embedded inside long JSX or compound service conditions, which reduces scanability, for example the long folder/task comparisons in 'backend/src/services/task.service.js' and inline view expressions in frontend components.
34	In logical expressions, have the following been addressed	☐	☒	☐	Partially only. The codebase contains many valid boolean checks, but there are still purpose/readability issues and multi-clause conditions that do not meet the stricter checklist style; see sub-items 34a-34e.
	a. Does the logical expression serve the purpose for which it is used?	☐	☒	☐	Not always. The translated-label logic in 'frontend/app/components/ProfileSettings.tsx' is a concrete case where the logical branch no longer serves the intended business purpose under non-English labels.
	b. Does each relational expression used result in a true or false outcome ?	☒	☐	☐	
	c. Is each relational expression inside a set of parentheses?	☐	☒	☐	No. Multi-part expressions often rely on operator precedence without wrapping each relation, for example 'if (sourceTask.folderId && linkedTask.folderId && sourceTask.folderId.toString() !== linkedTask.folderId.toString())' in 'backend/src/services/task.service.js'.
	d. At any given time, are only two relational expressions compared?	☐	☒	☐	Several logical conditions chain more than two relational/truthy checks at once, such as the folder-link validation and permission checks in 'backend/src/services/task.service.js'.
34	e. Is there any need for horizontal scrolling to read the entire expression?	☐	☒	☐	Some logical expressions become long enough to reduce readability, especially in 'backend/src/services/task.service.js' and large frontend JSX conditionals.
35	Have resource handling and cleanup been implemented properly?	☐	☒	☐	Partially only. Many listeners/timers/sockets are cleaned up correctly, but not all transient resources are consistently released; see sub-items 35a-35b.
	a. Are database connections, file streams, sockets, or external resources opened only when they are actually needed?	☒	☐	☐	
	b. Are these resources properly closed, released, or cleaned up after the operations are completed?	☐	☒	☐	Not consistently. Many cleanups exist, but 'frontend/app/components/ProfileSettings.tsx' creates 'avatarPreview' with 'URL.createObjectURL(file)' and no corresponding 'URL.revokeObjectURL(...)' was found, unlike the cleanup implemented in 'NotesView.tsx' and 'TaskDetailModal.tsx'.
36	In variable declarations, have the following been addressed:	☐	☒	☐	Partially only. There are clear issues with unnecessary declarations, naming consistency, and hard-coded values; see sub-items 36a-36d.
	a. Do all the variables declared as global or static really need to be global or static?	☒	☐	☐	
	b. Are there any declarations of unnecessary variables?	☐	☒	☐	Frontend linting reported many unnecessary declarations, for example 'setEmail' in 'frontend/app/components/ProfileSettings.tsx', 'canDeleteFolders'/'canEditFolders'/'canAssignFolders' in 'frontend/app/components/layouts/Sidebar.tsx', and 'selectedUser' in 'frontend/app/admin/page.tsx'.
	c. Do variable, function, class, and route names avoid reserved keywords and remain clear, consistent, and domain-meaningful?	☐	☒	☐	Not consistently. The reviewed scope already shows naming inconsistency such as 'Task.model.js'/'User.model.js' versus lowercase controller files, and some names are generic or mixed-language rather than consistently domain-standardized.
	d. Is there any hard coding inside the code?	☐	☒	☐	Hard-coded values remain in several places, for example the default super-admin credentials in 'backend/src/utlis/initSuperAdmin.js', hard-coded localhost fallback URLs in frontend components/services, and literal labels such as "Time Zone" in 'ProfileSettings.tsx'.
37	Do controllers remain thin and delegate business logic to service or utility layers?	☒	☐	☐	
38	Have route handlers been organized consistently according to the project structure?	☒	☐	☐	
39	Are request inputs validated before being processed?	☐	☒	☐	Partially only. The project has substantial validation support, but it is not applied consistently at the route boundary for every input path; see sub-items 39a-39d.
	a. Are req.body fields validated?	☐	☒	☐	Not consistently at the route layer. Some bodies are validated through middleware such as 'validateCreateTask', but other body-driven routes like 'POST /api/tasks/:id/link', 'POST /api/tasks/:id/repeat', and 'POST /api/tasks/:id/custom-status' in 'backend/src/routes/task.routes.js' do not apply dedicated request-body validator middleware before reaching the controller/service.
	b. Are req.params values validated?	☐	☒	☐	Not consistently at the route boundary. Several param-based routes such as 'GET /api/tasks/:id', 'POST /api/groups/:id/join', and 'GET /api/groups/:id/tasks' do not use explicit param validator middleware before controller execution.
	c. Are req.query values validated?	☐	☒	☐	Not consistently before processing. Query-heavy endpoints rely on service-level sanitation in some places, but there is no uniform query-validator middleware applied across list/search endpoints at the route layer.
	d. Are invalid ObjectIds values rejected safely?	☒	☐	☐	
40	Are authorization and access control checks implemented correctly?	☒	☐	☐	
	a. Is authentication required on protected routes?	☒	☐	☐	
	b. Are role-based checks enforced for admin-only features?	☒	☐	☐	
	c. Are group-, folder-, or task-scoped permissions checked before access?	☒	☐	☐	
	d. Is access denied by default when permission is unclear or missing?	☒	☐	☐	
	Has error handling been implemented consistently across asynchronous operations?	☐	☒	☐	Partially only. The backend has shared 'sendSuccess'/'sendError' helpers in 'backend/src/utlis/response.js', but internal details are still exposed in some error paths, so response handling is not consistently safe across all API outcomes.
	a. Do success responses follow a consistent structure?	☒	☐	☐	
	b. Do error responses follow a consistent structure?	☒	☐	☐	

41	c. Are HTTP status codes used appropriately?	☒	☐	☐	
	d. Are internal error details hidden from end users?	☐	☒	☐	'backend/src/middlewares/errorHandler.js' adds 'response.stack = err.stack' in development, and 'backend/src/middlewares/auth.js' returns raw token-verification error messages via 'sendError(res, error.message, 401)', exposing internal detail instead of consistently hiding it.
42	Has error handling been implemented consistently across asynchronous operations?	☒	☐	☐	
	a. Are async route handlers wrapped safely?	☒	☐	☐	
	b. Are rejected promises handled properly?	☒	☐	☐	
	c. Are custom errors distinguished from unexpected system errors?	☒	☐	☐	
	d. Are errors logged at appropriate points?	☒	☐	☐	
43	Has duplicate business logic been avoided across controllers, services, and utilities?	☐	☒	☐	Similar chat business flows are duplicated across the group-chat and direct-chat stacks, for example parallel create/edit/delete/reaction/upload logic in 'backend/src/controllers/chat.controller.js' and 'backend/src/controllers/directChat.controller.js' with matching service layers 'chat.service.js' and 'directChat.service.js', increasing maintenance risk.
44	Are secrets and environment-specific values handled safely?	☐	☒	☐	The environment config still contains insecure hard-coded fallback secrets and permissive environment fallbacks; see 'backend/src/config/environment.js'.
	a. Are secrets excluded from source code?	☐	☒	☐	'backend/src/config/environment.js' embeds fallback secrets such as 'default_secret_change_this' and 'default_refresh_secret_change_this' directly in source code.
	b. Are environment variables used instead of hard-coded credentials?	☐	☒	☐	Not consistently. The JWT configuration in 'backend/src/config/environment.js' falls back to hard-coded credential strings when env vars are missing, instead of failing closed.
	c. Are fallback values safe and appropriate?	☐	☒	☐	Unsafe defaults remain, including 'mongodb://localhost:27017/todolist', hard-coded JWT fallback secrets, and permissive production-origin fallbacks in 'backend/src/app.js' / 'backend/src/realtime/server.js' that can allow all origins when explicit config is absent.
45	Are MongoDB and Mongoose queries designed safely and efficiently?	☐	☒	☐	Partially only. Indexes and pagination exist in many places, but several task queries still fetch and populate much more data than required, which weakens efficiency.
	a. Are frequently filtered fields indexed appropriately?	☒	☐	☐	
	b. Are queries limited to only required fields where possible?	☐	☒	☐	Several high-traffic task queries return broad documents instead of restricting fields, for example 'backend/src/services/task.service.js' repeatedly fetches full task documents and only uses 'populate(...)' without a matching top-level '.select(...)' for list/detail payload minimization.
	c. Is unnecessary populate usage avoided?	☐	☒	☐	'backend/src/services/task.service.js' contains many heavy populate chains such as 'createdBy', 'assignedTo.userId', 'comments.user', 'timeEntries.user', 'scheduledWork.user', 'activeTimers.userId', 'linkedTasks.*', and 'groupId' even on flows where only part of that data is required.
	d. Are full-collection scans avoided where possible?	☒	☐	☐	
	e. Are pagination limits applied to list endpoints?	☒	☐	☐	
46	Are schema definitions and model constraints aligned with business rules?	☐	☒	☐	Partially only. Core schemas define many required fields and constraints correctly, but some role-related behavior in the group model is misaligned with the intended business rules.
	a. Are required fields marked correctly?	☒	☐	☐	
	b. Are enums and defaults defined correctly?	☐	☒	☐	Not fully. 'backend/src/models/Group.model.js' still exposes role helper methods 'isProductOwner()' and 'isAdmin()' that always return 'false', which conflicts with the surrounding role-based business model and makes the implemented defaults/role semantics unreliable.
	c. Are unique fields enforced where required?	☒	☐	☐	
	d. Are timestamps or audit fields used consistently?	☒	☐	☐	
47	Are update and delete operations protected against invalid or unsafe behavior?	☐	☒	☐	Partially only. Missing-document and permission handling are generally present, but validation is not enforced consistently on partial updates before persistence.
	a. Are missing documents handled correctly?	☒	☐	☐	
	b. Are partial updates validated before persistence?	☐	☒	☐	Not consistently. Some updates use 'runValidators: true', but others do not, for example 'backend/src/services/user.service.js' updates notification settings and deactivation state with 'findByIdAndUpdate(..., { new: true })' without enabling schema validators.
	c. Are dangerous mass updates or deletes avoided?	☒	☐	☐	
	d. Are ownership or permission checks performed before modification?	☒	☐	☐	
	Have concurrency-sensitive operations been handled appropriately?	☐	☒	☐	Several multi-step write paths remain vulnerable to race conditions and partial success because they rely on read-modify-save flows without atomic protection or transactions.
	a. Are race conditions considered in assignment, notification, or status updates?	☐	☒	☐	'backend/src/services/task.service.js' timer start/stop loads a task, mutates 'activeTimers'/'timeEntries', and saves it back, so concurrent requests can overwrite each other or bypass duplicate-timer checks.
	b. Are atomic updates used where appropriate?	☐	☒	☐	Not consistently. Concurrency-sensitive flows such as timer mutations and linked-task updates in 'backend/src/services/task.service.js' use document mutation plus 'save()' instead of atomic operators like '\$push', '\$pull', conditional updates, or version-guarded writes.

48	c. Are transactions used where multiple dependent writes must succeed together?	☐	☒	☐	Multi-write operations such as invitation acceptance in `backend/src/services/notification.consumer.js` update `group`, `notification`, and `user` separately, and linked-task synchronization in `task.service.js` also performs dependent writes without a transaction.
49	Are file upload and attachment operations handled safely?	☐	☒	☐	Partially only. File type/size controls and filename sanitization exist, but upload-failure handling is not wired consistently across upload routes.
	a. Are file type checks implemented?	☒	☐	☐	
	b. Are file size limits enforced?	☒	☐	☐	
	c. Are upload failures handled correctly?	☐	☒	☐	Not consistently. `backend/src/routes/chat.routes.js` correctly attaches `handleMulterError`, but upload routes such as `backend/src/routes/task.routes.js` (`/:id/attachments`, `/:id/comments/with-file`) and `backend/src/routes/user.routes.js` (`/me/avatar/upload`) apply multer middleware without the dedicated upload error handler.
50	d. Are unsafe file names or paths sanitized?	☒	☐	☐	
	Are real-time Socket.IO events implemented consistently and securely?	☐	☒	☐	Partially only. Event naming and listener cleanup are mostly reasonable, but some room-join and meeting notification flows lack proper authorization checks.
	a. Are event names clear and consistent?	☒	☐	☐	
	b. Are room join and leave operations validated?	☐	☒	☐	Not consistently. `backend/src/services/chat.socket.js` validates membership before `chat:join`/`direct:join`, but `backend/src/services/meeting.socket.js` allows `meeting:join` to join a group/direct meeting room based only on IDs in the payload without verifying that the caller belongs to that group or conversation.
51	c. Are permission checks applied before emitting sensitive events?	☐	☒	☐	In `backend/src/services/meeting.socket.js`, the first participant can trigger `meeting:incoming-call` emissions to group members or direct participants after only looking up `groupId`/`conversationId`, without first enforcing the caller's authorization for that target.
	d. Are duplicate listeners avoided?	☒	☐	☐	
	e. Are listeners cleaned up properly when no longer needed?	☒	☐	☐	
	Are notification and messaging flows implemented reliably?	☐	☒	☐	Partially only. Recipient scoping and duplicate consolidation exist, but unread-count and reconnect handling are not reliable across the reviewed frontend/backend notification flow.
52	a. Are notifications sent only to authorized recipients?	☒	☐	☐	
	b. Are unread counts updated consistently?	☐	☒	☐	`frontend/app/components/NotificationDropdown.tsx` registers a socket listener that depends only on `socket` but reads `notifications.some(...)` from a stale closure, so duplicate detection and unread counts can drift after subsequent state updates.
	c. Are offline and reconnect scenarios handled appropriately?	☐	☒	☐	Not fully. `frontend/app/hooks/useSocket.ts` reconnects the socket, but `NotificationDropdown.tsx` only reloads notifications on mount and on new-event receipt, so missed notifications/unread state are not resynchronized automatically after reconnect.
	d. Are duplicate notifications prevented where possible?	☒	☐	☐	
53	Are logs and audit-related operations handled appropriately?	☐	☒	☐	Partially only. Important actions are logged, but sensitive credentials/tokens are still written to logs in the reviewed backend/frontend auth setup.
	a. Are important security or admin actions logged?	☒	☐	☐	
	b. Are sensitive values excluded from logs?	☐	☒	☐	Sensitive data is still logged, for example `backend/src/utlils/initSuperAdmin.js` prints the default super-admin password, `frontend/app/context/AuthContext.tsx` logs the access token after login, and `frontend/app/services/api.client.ts` logs the full request config that includes the `Authorization` header.
	c. Are log messages clear enough for debugging and tracing?	☒	☐	☐	
54	Are authentication and session-related flows implemented securely?	☐	☒	☐	Partially only. Token validation and inactive-account blocking are present, but refresh-token handling is not implemented safely.
	a. Are access tokens validated correctly?	☒	☐	☐	
	b. Are refresh token operations handled safely?	☐	☒	☐	Refresh tokens are stored in plaintext in both the backend and frontend, for example `User.refreshToken` is persisted directly by `backend/src/services/auth.service.js` and `frontend/app/services/auth.service.ts` stores the refresh token in `localStorage` instead of using a safer httpOnly-cookie or hashed-token approach.
	c. Are locked or inactive accounts blocked correctly?	☒	☐	☐	
55	d. Are password-related operations protected and validated properly?	☒	☐	☐	
	Are frontend service calls and UI logic separated appropriately?	☐	☒	☐	Partially only. Service modules exist, but API orchestration and mixed UI/business responsibilities are still embedded in large frontend components.
	a. Is API calling logic kept out of purely presentational components?	☐	☒	☐	frontend/app/components/AuthPages.tsx directly orchestrates auth-service requests in the UI component, and frontend/app/components/ProfileSettings.tsx performs avatar upload through a direct `fetch(...)` call instead of delegating all remote operations to a service/helper layer.
	b. Are reusable calls placed in service or utility modules?	☒	☐	☐	
56	c. Are large components split when they contain too much mixed responsibility?	☐	☒	☐	Large components such as `frontend/app/components/layouts/Sidebar.tsx` and `frontend/app/components/AuthPages.tsx` still combine substantial UI rendering, permission logic, state coordination, and request orchestration in one place.
	Are loading, empty, and error states handled properly in the frontend?	☐	☒	☐	Partially only. Loading and empty states are commonly present, but some request failures are only logged and do not expose a recoverable user-facing error state.

55	a. Is a loading state shown during async fetches?	☒	☐	☐	
	b. Is an empty state shown when no data exists?	☒	☐	☐	
	c. Is an error state shown when requests fail?	☐	☒	☐	Not consistently. `frontend/app/components/NotificationDropdown.tsx` catches notification-load failures with `console.error(...)` only, so the user receives no visible error state for that failed request path.
	d. Can the user recover from the error state where appropriate?	☐	☒	☐	Not consistently. In failure paths like notification loading, the reviewed UI offers no retry or resync action, leaving recovery dependent on a full page reload or a future socket event.
56	Is frontend state management implemented cleanly and safely?	☐	☒	☐	Mostly structured, but not fully safe. Contexts/hooks are used well overall, yet stale-state handling is still broken in at least one async notification flow.
	a. Is shared state placed in appropriate context or hooks?	☒	☐	☐	
	b. Is unnecessary prop drilling avoided?	☒	☐	☐	
	c. Are side effects isolated appropriately in hooks or effect handlers?	☒	☐	☐	
57	d. Are stale state and race conditions considered in async UI updates?	☐	☒	☐	`frontend/app/components/NotificationDropdown.tsx` uses socket callbacks that read captured `notifications` state rather than guaranteed-latest state, so async notification updates can become inconsistent.
	Are role-based UI restrictions implemented consistently on the frontend?	☒	☐	☐	
	a. Are admin-only controls hidden from non-admin users?	☒	☐	☐	
	b. Are group- or folder-scoped actions hidden when the user lacks permission?	☒	☐	☐	
58	c. Is restricted functionality still protected on the backend, not only in the UI?	☒	☐	☐	
	Are form handling and client-side validation implemented appropriately?	☒	☐	☐	
	a. Are required fields validated before submission?	☒	☐	☐	
	b. Are invalid formats rejected clearly?	☒	☐	☐	
59	c. Are validation messages understandable to users?	☒	☐	☐	
	d. Is server-side validation still enforced even when client-side validation exists?	☒	☐	☐	
	Have i18n, formatting, and user preference settings been handled correctly?	☐	☒	☐	Partially only. Shared preference contexts exist, but strings, date formatting, and language-preference handling are not applied consistently across the reviewed frontend.
	a. Are UI strings externalized where appropriate?	☐	☒	☐	Many strings remain hard-coded outside the shared i18n layer, for example `frontend/app/admin/page.tsx` contains numerous inline English labels, and `frontend/app/components/AuthPages.tsx` keeps its own local translation table instead of using the shared app translation system consistently.
60	b. Are date and time displays consistent with configured preferences?	☐	☒	☐	Several frontend views still format dates/times directly with `toLocaleDateString()` / `toLocaleString()` instead of consistently routing through regional-preference helpers, for example in `frontend/app/admin/page.tsx`, `components/views/TasksView/ChecklistSection.tsx`, and `RepeatTaskModal.tsx`.
	c. Are language and theme preferences loaded safely?	☐	☒	☐	Not consistently. Contexts such as `LanguageContext` and `RegionalContext` load preferences, but `frontend/app/components/AuthPages.tsx` keeps a separate local `language` state and translation map, so preference handling is fragmented rather than safely unified across the UI.
	Is the code aligned with the documented project architecture and module boundaries?	☐	☒	☐	Mostly aligned at the folder-structure level, but not fully at the module-boundary level. The project follows the intended broad architecture while some large frontend components still accumulate too many cross-module responsibilities.
	a. Do backend modules follow the intended controller-service-model separation?	☒	☐	☐	
60	b. Do frontend modules follow the intended app/component/service structure?	☒	☐	☐	
		☐	☒	☐	Components such as `frontend/app/components/layouts/Sidebar.tsx` and `frontend/app/components/AuthPages.tsx` still pull in many contexts, services, permission helpers, and UI responsibilities at once, making their module boundaries harder to understand and maintain.
	c. Are cross-module dependencies kept reasonable and understandable?				
Conclusion	Fully Approved		☐		
	Conditionally Approved		☐		
	Rejected		☒		
Discussion	The reviewed codebase shows some positive structure, including partial controller-service separation, existing validation support, shared response helpers, and generally organized frontend/backend modules. However, the review identified a large number of defects, with 85 failed checklist items and several issues affecting security, reliability, maintainability, and performance. Major risks include hard-coded secrets, unsafe refresh-token storage, inconsistent input validation, exposed internal error details, race-condition-prone multi-step updates, and missing authorization checks in some real-time flows. The frontend also contains oversized components, duplicated logic, stale-state issues, and inconsistent i18n/error-state handling. In addition, the recorded performance evidence indicates that current response-time expectations are not met. Overall, the code requires significant correction and refactoring before it can be accepted for approval.				